The simple Primitive equation model

Consider the Boussinesq equations in hydrostatic approximation

$$\partial_t \boldsymbol{u} + f\underset{\neg}{\boldsymbol{u}} + \boldsymbol{\nabla} p = -\boldsymbol{u}\cdot\boldsymbol{\nabla}\boldsymbol{u} - w\partial_z \boldsymbol{u} + \boldsymbol{F}(\boldsymbol{u}) \quad , \quad \partial_t b = -\boldsymbol{u}\cdot\boldsymbol{\nabla} b - w\partial_z b + Mb \tag{1}$$

with the diagnostic relations $\partial_z p = b$ and $\boldsymbol{\nabla}\cdot\boldsymbol{u} + \partial_z w = 0$. Vectors are two-dimensional and $\underset{\neg}{\boldsymbol{u}}$ denotes anticlockwise rotation by 90°. Pressure is scaled with background density ρ_0 and thermodynamics have been simplified to a buoyancy equation. $\boldsymbol{F}$ and M are the friction and mixing operators. At the free surface $z = \eta$, we also have the condition

$$\partial_t p_s = -g\boldsymbol{\nabla}\cdot\int_{-H}^{\eta} \boldsymbol{u} dz \tag{2}$$

with $p_s = p|_{z=\eta} = g\eta$. Alternatively, we use a rigid lid with $w = 0$ at $z = 0$ and $p_s = p|_{z=0}$. We split full pressure p into hydrostatic part and surface pressure.

$$p = p_h(z) + p_s \ , \ p_h = \int_{-h}^{z} b dz' \tag{3}$$

where p_s is to be determined by the external mode solvers.

Simple implicit external solver for rigid lid

Velocity and buoyancy and pressure are all co-located in time in this scheme. The buoyancy equation is integrated as

$$b^{n+1} - b^n = \Delta t \left(\overline{\delta b}^{n+1/2} + M(b^n) \right) \tag{4}$$

with the advective buoyancy tendency

$$\delta b^n = \boldsymbol{\nabla}\cdot\boldsymbol{u}^n b^n + \partial_z w^n b^n \quad , \quad \overline{\delta b}^{n+1/2} = A\delta b^n + B\delta b^{n-1} + C\delta b^{n-2} \tag{5}$$

interpolated with the Adam-Bashforth (AB3) scheme to time level $n + 1/2$. Mixing is treated with an Euler forward step, or maybe implicit in case of vertical mixing.

An intermediate velocity $\boldsymbol{u}^*$ is calculated from

$$\boldsymbol{u}^* - \boldsymbol{u}^n = \Delta t \left(\overline{\delta \boldsymbol{u}}^{n+1/2} + \boldsymbol{F}(\boldsymbol{u}^n) \right) \tag{6}$$

with the momentum tendency $\delta\boldsymbol{u}$ excluding the surface pressure gradient and friction

$$\delta\boldsymbol{u}^n = -f\underset{\neg}{\boldsymbol{u}}^n - \boldsymbol{\nabla} p_h^n - \boldsymbol{u}^n\cdot\boldsymbol{\nabla}\boldsymbol{u}^n - w^n\partial_z\boldsymbol{u}^n \tag{7}$$

These tendencies are extrapolated with AB3 to time level $n + 1/2$

$$\overline{\delta\boldsymbol{u}}^{n+1/2} = A\delta\boldsymbol{u}^n + B\delta\boldsymbol{u}^{n-1} + C\delta\boldsymbol{u}^{n-2} \tag{8}$$

Friction is treated with an Euler forward step. From the vertically integrated continuity equation follows the condition

$$\boldsymbol{\nabla} \cdot \int_{-h}^{0} \boldsymbol{u}^* = \Delta t \boldsymbol{\nabla} \cdot h \boldsymbol{\nabla} p_s^{n+1} \tag{9}$$

which is solved with a conjugate-gradient iteration for p_s^{n+1} (attributes `epsilon` for the convergence threshold and `max_iterations`; a BiCGStab variant is implemented as an alternative). The iteration count and the estimated error are available per step as `congr_itts` and `estimated_error`. The intermediate velocity is then corrected as

$$\boldsymbol{u}^{n+1} = \boldsymbol{u}^* - \Delta t \boldsymbol{\nabla} p_s^{n+1} \tag{10}$$

It is possible to include an implicit free surface in the time stepping. Vertical velocity is calculated from continuity

$$w^{n+1} = -\int_{-h}^{z} \boldsymbol{\nabla} \cdot \boldsymbol{u}^{n+1} \tag{11}$$

This scheme is also used as default in the MITgcm.

Simple split-explicit external mode solver

Velocity is known in this scheme at $t = n + 1/2$, while b and p at level n. The momentum equation is discretised as

$$\boldsymbol{u}^* - \boldsymbol{u}^{n-1/2} = \Delta t \left(-f \underline{\overline{\boldsymbol{u}}}^{n} + \overline{\delta \boldsymbol{u}}^{n} - \boldsymbol{\nabla} p_h^n - \boldsymbol{\nabla} p_s^n + \boldsymbol{F}(\boldsymbol{u}^{n-1/2}) \right) \tag{12}$$

with the intermediate velocity $\boldsymbol{u}^*$ and AB3 extrapolation of Coriolis force and advection terms tendencies

$$\underline{\overline{\boldsymbol{u}}}^{n} = A \underline{\boldsymbol{u}}^{n-1/2} + B \underline{\boldsymbol{u}}^{n-3/2} + C \underline{\boldsymbol{u}}^{n-5/2} \quad , \quad \overline{\delta \boldsymbol{u}}^{n} = A \delta \boldsymbol{u}^{n-1/2} + B \delta \boldsymbol{u}^{n-3/2} + C \delta \boldsymbol{u}^{n-5/2} \tag{13}$$

and

$$\delta \boldsymbol{u}^{n-1/2} = -\boldsymbol{u}^{n-1/2} \cdot \boldsymbol{\nabla} \boldsymbol{u}^{n-1/2} - w^{n-1/2} \partial_z \boldsymbol{u}^{n-1/2} \tag{14}$$

For the hydrostatic and surface pressure $p_h + p_s$ no extrapolation is made, since they are at level n anyways, and friction is treated with Euler forward.

Now the momentum tendencies are vertically integrated but without surface pressure and Coriolis contributions.

$$\boldsymbol{R}^n = \int_{-h}^{0} \left(\overline{\delta \boldsymbol{u}}^{n} - \boldsymbol{\nabla} p_h^n + \boldsymbol{F}(\boldsymbol{u}^{n-1/2}) \right) dz \tag{15}$$

The external mode is integrated with sub-cycling over $m = 1, ...M$ (M set by the attribute `sub_cycling`) from n to $n+1$ with a simple, dissipative scheme proposed by Demange et al (2019) using the forcing $\boldsymbol{R}^n$

$$\boldsymbol{U}^{n+(m+1)/M} - \boldsymbol{U}^{n+m/M} = \Delta t/M \left(-f\overline{\boldsymbol{U}}^{n+(m+1/2)/M} - h\boldsymbol{\nabla} p_s^{n+m/M} + \boldsymbol{R}^n \right) \quad (16)$$

$$p_s^{n+(m+1)/M} - p_s^{n+m/M} = -g\Delta t/M \left((1+\theta)\boldsymbol{\nabla}\cdot\boldsymbol{U}^{n+(m+1)/M} - \theta\boldsymbol{\nabla}\cdot\boldsymbol{U}^{n+m/M} \right) \quad (17)$$

with the transport $\boldsymbol{U}^n = \int_{-h}^{0} \boldsymbol{u}^n dz$, and $\overline{\boldsymbol{U}}^{n+(m+1/2)/M}$ interpolated with AB2. Demange et al (2019) found $\theta = 0.14$ to be the best choice in terms of phase error and damping. The new external mode is given by

$$\boldsymbol{U}^{n+1} = \frac{1}{M}\sum_{m=1}^{M} \boldsymbol{U}^{n+m/M} + \frac{\theta}{M}\left(\boldsymbol{U}^{n+1} - \boldsymbol{U}^n\right) \quad (18)$$

After the sub-cycling, the barotropic part of the intermediate velocity is corrected

$$\boldsymbol{u}^{n+1/2} = \boldsymbol{u}^* - 1/h \left(\int_{-h}^{0} \boldsymbol{u}^* dz - \boldsymbol{U}^{n+1} \right) \quad (19)$$

The buoyancy equation is integrated using $\boldsymbol{u}^{n+1/2}$ and $w^{n+1/2} = -\int_{-h}^{z} \boldsymbol{\nabla}\cdot\boldsymbol{u}^{n+1/2}$ as

$$b^{n+1} - b^n = \Delta t \left(\delta b^{n+1/2} + M(b^n) \right) \quad (20)$$

with the buoyancy tendency

$$\delta b^n = \boldsymbol{\nabla}\cdot\boldsymbol{u}^{n+1/2} b^n + \partial_z w^{n+1/2} b^n \quad (21)$$

interpolated with AB3 to time level $n + 1/2$. This scheme is one option in FESOM2. The external-mode treatment is selected by the attributes `implicit_free_surface` and `explicit_free_surface`: both false gives the rigid lid, the first the implicit free surface (both solved with the iterative solver above), the second the split-explicit scheme.

Vertical mixing and friction options

Vertical mixing of buoyancy and vertical friction are organised in a hierarchy of options similar to pyOM2, selected by the attribute `vmix_scheme` $\in$ {`'explicit'`, `'implicit'`} and two switches (module `implicit_vmix.py`; the previous flag `enable_implicit_vmix` maps to the implicit scheme with convective adjustment, so existing configurations are unchanged).

Explicit scheme. Fixed coefficients κ (buoyancy) and A_v (momentum) only, integrated forward in time,

$$b^{n+1} = b^n + \Delta t\, \partial_z \left(\kappa\, \partial_z b^n \right), \qquad \boldsymbol{u}^{n+1} = \boldsymbol{u}^{n+1} + \Delta t\, \partial_z \left(A_v\, \partial_z \boldsymbol{u} \right) \quad (22)$$

subject to the diffusive stability limit, which is checked at start-up (`check_vmix_stability`, called from `prepare_for_run`): $4\max(\kappa, A_v)\,\Delta t/\Delta z^2 \le 1/2$; the model refuses to run otherwise, and also refuses convective adjustment or the TKE closure under the explicit scheme.

Implicit scheme. Both operators are integrated with backward Euler (unconditionally stable, so arbitrarily large coefficients are allowed), as a tridiagonal solve per column with the coefficient located on the W points. Three sub-options for the coefficients: (i) constant κ and A_v; (ii) convective adjustment (`enable_conv_adjust`): background κ, but $\kappa = \kappa_{conv}$ (large) where $N^2 < 0$; (iii) the TKE closure (`enable_tke`), which overrides all constant coefficients for both buoyancy and momentum as described below. Vertical friction is applied after the surface pressure correction: since the vertical operator with zero-flux boundary conditions leaves the column-integrated momentum unchanged, the barotropic mode is not perturbed, and w is re-diagnosed afterwards.

TKE closure

The closure of Gaspar et al. (1990) is ported from pyOM2 (`tke.py`, mixin class). All closure constants are configuration attributes (set before `init_tke`); the defaults are the pyOM2 `tke_module` defaults: $c_k = 0.1$, $c_\epsilon = 0.7$, $\alpha = 1$, $\ell_{min} = 10^{-12}\,\mathrm{m}$, $\kappa_M^{max} = 100\,\mathrm{m^2/s}$, and background floors $\kappa_M^{min} = \kappa_H^{min} = 10^{-5}\,\mathrm{m^2/s}$ (the floors play the role of pe_simple's constant background A_v and κ; pyOM2 handles backgrounds separately and uses $\kappa_M^{min} = 0$). Note that the pyOM2 configurations of the acc family deviate from the module defaults and set $\alpha = 30$ (a proxy for vertical TKE transport by pressure work, which spreads the surface-injected TKE substantially deeper) and $\ell_{min} = 10^{-8}\,\mathrm{m}$; for consistency with those experiments both should be set accordingly (`alpha_tke`, `mxl_min`). The turbulent kinetic energy e is a prognostic variable on the W points,

$$\partial_t e = \partial_z\left(\alpha\,\kappa_M \partial_z e\right) + K_{diss} - P_{diss} - c_\epsilon\, e^{3/2}/\ell \tag{23}$$

with the mixing length $\ell = \sqrt{2e}/N$, limited such that it cannot grow faster than the distance to the boundaries (two sweeps, pyOM2's `tke_mxl_choice=2`), and the diffusivities

$$\kappa_M = \min(\kappa_M^{max},\, c_k\,\ell\sqrt{e}), \qquad \kappa_H = \kappa_M/\max(1, \min(10,\, 6.6\,Ri)), \qquad Ri = N^2\kappa_M/K_{diss} \tag{24}$$

Convection is contained in the closure: $\kappa_H = \max(\kappa_H, 1)$ where $N^2 < 0$, which replaces the convective-adjustment option.

The discretisation follows pyOM2: backward Euler with the dissipation linearised in the diagonal ($c_\epsilon\sqrt{e^n}/\ell$ acting on e^{n+1}), a surface W cell of half thickness $\Delta z/2$, and the surface flux boundary condition added to that cell,

$$F_e(0) = \begin{cases} \Phi_{oc} & \text{wave dissipation flux from the wave model} \\ (|\boldsymbol{\tau}|)^{3/2} & \text{classical parameterisation} \end{cases} \tag{25}$$

selected by `tke_surf_from_waves` (default on, falling back to the classical form if no wave flux is present; $\boldsymbol{\tau}$ kinematic; the first form closes the energy cycle between the wave and

ocean components, the second is the pyOM2 default and serves as the control experiment). No diffusive TKE, tracer or momentum flux passes through the surface face itself: the surface forcing enters only as the explicit flux above, which also removes a spurious shear production against the zero-velocity halo above the surface (a phantom no-slip lid). For wave-driven injection the downward mixing-length limiting sweep can be seeded with a surface roughness length (attribute `z0_surf`, default 0, e.g. $1.6\,H_s$ from the wave model in the sense of Mellor & Blumberg 2004), so the near-surface mixing length starts at the wave scale rather than at $O(\Delta z)$. If the surface cell is drained negative by buoyancy fluxes, the missing energy is added back and booked in `tke_surf_corr` as in pyOM2, and $e \geq e_{min} = 10^{-12}$ is enforced.

Energetic consistency is exact on the discrete level: vertical friction is computed implicitly with κ_M averaged to the U/V points as in pyOM2 ($\overline{\kappa_M}^i$), and the shear production is diagnosed from the same discrete operator with the updated velocities,

$$K_{diss} = \overline{\overline{\kappa_M}^i \left(\partial_z u^{n+1}\right)^2}^i + \overline{\overline{\kappa_M}^j \left(\partial_z v^{n+1}\right)^2}^j \tag{26}$$

so the kinetic energy removed from the mean flow reappears in the TKE forcing without discretisation residual; $P_{diss} = \kappa_H N^2$ is the exact transfer of the implicit buoyancy mixing, evaluated with the *updated* buoyancy (the tracer solve precedes the TKE step; using the pre-mixing N^2 over-drains the TKE whenever $\kappa_H \Delta t/\Delta z^2$ is not small). In stand-alone column tests the discrete TKE budget (surface flux + shear production = dissipation + buoyancy destruction + storage) closes to machine precision.

When the model velocity is the Lagrangian mean $\boldsymbol{u}_L$ of the coupled wave–ocean formulation, the attribute `mix_eulerian` (default off, requires the Stokes velocity to be present) lets the friction operator act on the Eulerian mean velocity $\bar{\boldsymbol{u}} = \boldsymbol{u}_L - \boldsymbol{u}_S$: the downgradient closure parameterises Eulerian fluctuations, so mixing $\boldsymbol{u}_L$ would spuriously diffuse the Stokes shear. The Stokes velocity is subtracted before the implicit solve and added back afterwards, and the shear-production booking becomes flux times *Lagrangian* shear, split into the non-negative Eulerian part $K^E_{diss} = \kappa_M(\partial_z \bar{\boldsymbol{u}})^2$ and the sign-indefinite Stokes (Langmuir) part $K^L_{diss} = \kappa_M \partial_z \bar{\boldsymbol{u}} \cdot \partial_z \boldsymbol{u}_S$, kept separately in the diagnostics with $K_{diss} = K^E_{diss} + K^L_{diss}$ feeding the TKE equation. Without `mix_eulerian` this reduces bit-identically to the plain booking above.

The calling sequence per time step is: diffusivities from the current e and N^2 at the start of the step (so buoyancy mixing and friction use the same coefficients the transfers are paired with), vertical friction after the pressure correction, and the TKE step at the end of the time step.

Model parameters and options

All parameters and options are attributes of the model instance, set either through the parameter dictionary passed at construction or by assigning the attribute before `prepare_for_run`; options with a default may be left unset. The tables list every attribute the model reads.

Required parameters (no default)

Attribute	Unit	Meaning
`nx`, `ny`, `nz`	—	global grid size (cells, without halo points)
`dx`, `dy`, `dz`	m	grid spacing; Δz is a uniform scalar
`dt`	s	time step
`eq_of_state`	—	equation of state: 0 (temperature carries the buoyancy, no salinity), 1 (linear $\rho(S,T)$, salinity prognostic), 100 (ideal-gas atmosphere)
`implicit_free_surface`	—	implicit free surface if true
`explicit_free_surface`	—	split-explicit free surface if true; both false: rigid lid
`periodic_in_x`, `periodic_in_y`	—	lateral boundaries periodic (true) or closed (false)
`epsilon`	—	convergence threshold of the iterative surface-pressure solver (rigid lid / implicit free surface)
`max_iterations`	—	iteration cap of that solver
`sub_cycling`	—	number of barotropic sub-cycles M (split-explicit scheme only)

The constructor additionally takes `n_pes_x`, `n_pes_y` (processor grid, default 1×1), `halo_points` (default 1) and `comm_in` (default: MPI world communicator). The constants `grav` = 9.81 and `rho_0` = 1024 are set internally (note: they overwrite any value passed in the parameter dictionary). Coriolis parameter, topography (`kbot`) and initial conditions are set through the template methods `set_coriolis`, `set_topography` and `set_initial_conditions`, overloaded in the configuration.

Forcing hooks and coupling fields

Attribute / method	Meaning
`set_forcing(tau)`	optional user method; adds forcing to the momentum tendencies `du[tau]`, `dv[tau]` (enters the AB interpolation)
`set_forcing_direct()`	optional user method; direct momentum increment bypassing the AB weights, applied before the pressure solve. Used for the Stokes tendency $\Delta \boldsymbol{u}_S$ of the Lagrangian-mean ($\boldsymbol{u}_L$) formulation, which must be a plain forward increment
`taux`, `tauy`	kinematic wind stress; also the fallback surface TKE forcing $\lvert\boldsymbol{\tau}\rvert^{3/2}$
`phi_oc`	wave-breaking energy flux from the wave model; surface TKE flux when `tke_surf_from_waves` is on

Attribute / method	Meaning
`u_S`, `v_S`	Stokes velocity from the wave model; required by `mix_eulerian`

Options with defaults

Attribute	Default	Meaning
`superbee_advection`	unset	if set true, tracer advection uses the Superbee flux limiter instead of the second-order fluxes (momentum advection is always second order). Note: setting it to false raises an error; leave it unset for the second-order scheme
`Khbi`	unset	biharmonic horizontal tracer diffusion coefficient (m^4/s); active if present, forward Euler
`Ahbi`	unset	biharmonic horizontal friction coefficient (m^4/s); active if present, forward Euler (rigid-lid/implicit path)
`vmix_scheme`	`'explicit'`	vertical mixing/friction hierarchy: `'explicit'` (forward Euler, coefficient checked against the diffusive stability limit) or `'implicit'` (backward Euler, unconditionally stable). If unset, the legacy flag `enable_implicit_vmix` decides
`kappa_T`	0	background vertical diffusivity (m^2/s)
`A_v`	0	background vertical viscosity (m^2/s)
`enable_conv_adjust`	on iff implicit	convective adjustment: $\kappa = \kappa_{conv}$ where $N^2 < 0$ (implicit scheme only)
`kappa_conv`	1	convective-adjustment diffusivity (m^2/s)
`enable_tke`	off	TKE closure; overrides the constant coefficients for both buoyancy and momentum (implicit scheme only)

TKE closure constants (read at `init_tke`)

Attribute	Default	Meaning
`c_k`	0.1	diffusivity constant, $\kappa_M = c_k \ell \sqrt{e}$
`c_eps`	0.7	dissipation constant, $c_\epsilon e^{3/2}/\ell$

Attribute	Default	Meaning
`alpha_tke`	1	vertical TKE diffusion $\alpha\kappa_M$; the pyOM2 acc configurations use 30
`mxl_min`	10^{-12}	mixing-length floor (m); acc configurations use 10^{-8}
`kappaM_min`	10^{-5}	viscosity floor (m^2/s), plays the role of the constant background A_v
`kappaM_max`	100	viscosity cap (m^2/s)
`kappaH_min`	10^{-5}	diffusivity floor (m^2/s)
`tke_min`	10^{-12}	TKE floor (m^2/s^2)
`z0_surf`	0	surface roughness length (m) seeding the downward mixing-length sweep (e.g. $1.6\,H_s$)
`tke_surf_from_waves`	on	surface TKE flux from the wave dissipation Φ_{oc}; falls back to $\lvert\boldsymbol{\tau}\rvert^{3/2}$ if no wave flux is present
`mix_eulerian`	off	vertical friction acts on the Eulerian mean velocity $\boldsymbol{u}_L - \boldsymbol{u}_S$ ($\boldsymbol{u}_L$ formulation), with the K^E_{diss}/K^L_{diss} booking split

Snapshot diagnostics

Two snapshot classes are available; either replaces the model class as the base of the configuration and overrides `prepare_for_run` and `diagnose`. Both read the attribute `snapint` (snapshot interval in seconds, required) and take the `prepare_for_run` arguments `snap_file_name` and `show_each_time_step` (default on: print every time step; off: print at output steps only). When the iterative surface-pressure solver is active, both print its per-step iteration count and accumulate the total count between outputs.

snap_nc4 writes a single netCDF file (default name `'snap.cdf'`), created with the T- and U-grid coordinate axes and the variables u, v, w, temp, p_h, ρ and p_s; salinity is included when the equation of state carries it (`eq_of_state` > 0 and $\neq 100$). At every output step each PE in turn writes its halo-free sub-block into the global arrays of the shared file (serialised with barriers). For the rigid-lid/implicit path the current time level of the surface pressure is written, for the split-explicit path the plain 2D field. This class works with the MPI domain decomposition and with single-process JAX, but not in a multihost JAX environment, since all processes must be able to open the same file.

snap_raw writes one `.npy` file per variable, per PE and per output step into a directory tree `<name>/pe=<rank>/n=<step>/` (default name `'raw_snap'`; an existing directory of that name is removed at start-up), with the coordinate axes and the processor-grid layout (`decomposition.npy`) stored once at the top level. Each process only touches its own subdirectory, and JAX sharded arrays are written through their local shard, so this class also works in a multihost JAX environment. The variable set is u, v, w, temp, salt, p_h and p_s (salt always, ρ not included). The fragmented output is meant to be reassembled

afterwards using the stored decomposition, e.g. into zarr.